

Yes. Below is the **first planned report** for the project. Treat this as the blueprint we will edit before writing any code.

Automated Earnings Quality Analyst

Project Development Plan — Version 1

1. Project goal

Build an **auditable financial-analysis agent** that can analyze a public company using the five Week 5 learning objectives:

1. Accrual vs. cash accounting
2. D&A, CapEx, and working-capital effects
3. Deferred taxes and stock-based compensation
4. Normalized earnings and one-off adjustments
5. Earnings-quality red flags

The user should be able to enter a company/ticker and receive:

Financial data → calculations → red flags → explanations → Excel report → final analyst conclusion

The important rule is that every conclusion must be **traceable back to the source data and calculations**.

2. Core principle

The agent should not simply say:

“Tesla has weak earnings quality.”

It should show:

Step 1 — Source data

Revenue = X

Net Income = X

OCF = X

AR = X

Inventory = X

etc.

↓

Step 2 — Calculations

Revenue growth
AR growth
Inventory growth
NI growth
OCF growth
FCF
SBC ratios
etc.

↓

Step 3 — Rules / flags

Example:

AR growth = 35%
Revenue growth = 10%

→ **Flag: AR growing materially faster than revenue**

↓

Step 4 — Interpretation

Possible slower collections, looser credit terms, aggressive revenue recognition, etc.

↓

Step 5 — Analyst conclusion

Strong / Moderate / Weak earnings quality.

This allows us to check the agent at **every stage** instead of blindly trusting AI.
maby put all the values in a table that can show case the calculations whith charts realitive to them

3. Scope of Version 1

Version 1 should analyze:

Public non-financial companies initially.

We should avoid banks and insurers at the beginning because their financial statements and working-capital logic are different.

Target analysis period:

3–5 fiscal years

Initial data required:

- Revenue
- Net Income
- Operating Cash Flow
- Accounts Receivable
- Inventory
- Accounts Payable
- D&A
- CapEx
- Stock-Based Compensation
- Shares Outstanding
- Tax Expense
- Deferred Tax Assets
- Deferred Tax Liabilities
- unusual gains/losses
- restructuring charges
- impairments
- asset-sale gains
- other potential one-offs

4. Module 1 — Data Extraction

Goal

Automatically retrieve financial information for the selected company.

User enters:

AAPL

Agent identifies:

Apple Inc.

Then obtains financial data for the required years.

What must be stored

Each data point should contain:

- Company
- Fiscal year
- Metric
- Value
- Units
- Financial statement
- Source
- Source date

Example:

Metric	Year	Value	Statement
Revenue	2025	...	Income Statement
Net Income	2025	...	Income Statement
Accounts Receivable	2025	...	Balance Sheet
OCF	2025	...	Cash Flow

Accuracy test

Before continuing, we manually compare extracted numbers with the original filing.

Target:

100% match for the core financial figures.

If the extraction is wrong, the agent should not proceed.
we should insure that the agent get the info from reliable sites from the companys websites

5. Module 2 — Data Validation

This is one of the most important parts.

The system checks for:

- missing years
- different units

- duplicated numbers
- negative values that may be formatting issues
- changed fiscal years
- missing financial items
- inconsistent labels

Example:

If one source reports:

Revenue = 95,000

but the filing says units are **USD millions**, the program must understand that this means:

\$95 billion

We should also allow a manual correction before analysis.

6. Module 3 — Accrual vs Cash Analysis

This covers Week 5 Objective 1.

Calculate:

Revenue growth

Current Revenue / Previous Revenue - 1

AR growth

Current AR / Previous AR - 1

Net Income growth

OCF growth

Then compare:

Revenue vs AR

and

Net Income vs OCF

Potential flags:

AR growing much faster than Revenue

→ collections may be weakening.

NI rising while OCF falls

→ reported profit may not be converting into cash.

The manual specifically identifies profit/cash differences and receivables as important earnings-quality issues.

7. Module 4 — D&A, CapEx and Working Capital

This handles Objective 2.

The system analyzes:

D&A

Compare:

D&A trend

D&A / Revenue

CapEx

Calculate:

$$\text{FCF} = \text{OCF} - \text{CapEx}$$

Compare:

CapEx vs D&A

But do NOT automatically say high CapEx is bad.

Agent should classify it as:

Needs investigation

and ask whether CapEx is:

- maintenance
- expansion
- new factories
- technology investment
- acquisition-related

Working capital

Analyze:

AR ↑ → **cash use**

Inventory ↑ → **cash use**

AP ↑ → **cash benefit**

Then compare working-capital growth to business growth.

8. Module 5 — Deferred Taxes

This covers the first half of Objective 3.

Extract:

- DTA
- DTL
- current tax expense
- deferred tax expense
- tax-loss carryforwards where available

The system explains:

DTL

Tax benefit today / expected future tax obligation.

DTA

Potential future tax benefit.

Important DTA check:

Does the company appear capable of generating enough taxable income to use the DTA?

The system should **not automatically value a DTA at 100%**.

9. Module 6 — Stock-Based Compensation

Analyze:

- SBC
- SBC growth
- SBC / Revenue
- SBC / OCF
- share-count growth
- share repurchases

The agent should distinguish between:

Non-cash accounting treatment

and

economic cost to shareholders.

Example:

SBC ↑ 40%

Share count ↑ 8%

→ **possible significant dilution**

Another case:

SBC ↑ 40%

Share count flat

Large buybacks

→ dilution controlled, but investigate how much cash was required for buybacks.

10. Module 7 — Earnings Normalization

This handles Objective 4.

Start with:

Reported Net Income

Then identify possible one-offs.

Examples:

- asset-sale gains
- restructuring expenses
- legal settlements
- impairment charges
- acquisition-related expenses
- unusual tax gains
- other unusual items

Basic calculation:

Reported NI

– genuine one-time gains

- genuine one-time expenses

=

Normalized Net Income

But the agent must also review historical frequency.

Example:

“Restructuring expense”

2022 

2023 

2024 

2025 

The program should flag:

This appears recurring and should not automatically be treated as a one-off.

That judgment is critical.

11. Module 8 — Earnings Quality Red-Flag Engine

This combines all previous modules.

Possible signals:

Cash quality

NI ↑ while OCF ↓

Receivables

AR growth >> Revenue growth

Inventory

Inventory growth >> Revenue growth

Payables

AP rising unusually fast while liquidity is weak

SBC

SBC growing rapidly + share dilution

Adjustments

Repeated “one-time” charges

Normalization

Large difference between reported and normalized NI

Taxes

Large DTA with weak profitability

The agent should never conclude:

“Fraud.”

Instead:

**Low risk
Needs investigation
Material concern**

after all the model we should be able to see them clearly in a table

12. Earnings Quality Score

Eventually we can create a scoring system.

Example:

0–100 Earnings Quality Score

Possible categories:

80–100 → Strong

60–79 → Generally healthy

40–59 → Needs investigation

Below 40 → Weak

But we should **not build the score first**.

First we build and validate individual rules.

Only after the rules work properly should we combine them into a score.

13. AI Interpretation Layer

AI comes **after the calculations**.

AI receives structured results such as:

Revenue growth: 12%

AR growth: 38%

NI growth: 20%

OCF growth: -5%

Instead of calculating them itself.

AI then explains:

Receivables are growing substantially faster than revenue while operating cash flow is declining despite higher reported earnings. This suggests weakening cash conversion and warrants investigation into collection periods and revenue-recognition practices.

This reduces hallucination risk.

14. Excel Report Generator

The system automatically generates:

`CompanyName_Earnings_Quality_Analysis.xlsx`

Suggested sheets:

1. Company Overview

2. Raw Financial Data

3. Growth Calculations

4. Accrual & Cash Analysis

5. Working Capital

6. D&A and CapEx

7. Deferred Taxes

8. SBC & Dilution

9. Normalized Earnings

10. Red Flags

11. Dashboard

12. Analyst Conclusion

15. Excel Charts

Useful charts:

Net Income vs Operating Cash Flow

Revenue growth vs AR growth

Revenue growth vs Inventory growth

D&A vs CapEx

OCF vs FCF

SBC trend

Shares outstanding trend

Reported NI vs Normalized NI

We should not create charts simply because Excel allows us to. Every chart should answer an analytical question.

16. Pivot Tables

Pivot tables should only be added when useful.

They become especially valuable later when we analyze:

- multiple companies
- multiple quarters
- multiple business segments

- industries
- peer groups
- or building clean charts

For one company with five annual observations, normal tables are usually cleaner.

17. Streamlit Interface

Your current portfolio already uses Streamlit and Python, so we can eventually integrate this project with that environment.

Possible interface:

Earnings Quality Analyzer

Company ticker

[AAPL]

Years

[5]

Analyze Company

Then tabs:

Overview

Financial Data

Cash Conversion

Working Capital

Taxes

SBC

Normalized Earnings

Red Flags

AI Analysis

Download Excel

18. Accuracy System

This should be a major feature of the project.

For every result, show:

Source

Where did the number come from?

Raw value

What was extracted?

Calculation

What formula was used?

Result

What did Python calculate?

Rule

Why was something flagged?

Interpretation

What did AI conclude?

Therefore:

Source → Data → Formula → Result → Flag → Interpretation

This is the backbone of the whole application.

19. Testing Plan

We should build using **one test company first**.

I suggest eventually picking a company with reasonably clear statements.

Then manually calculate every Week 5 item ourselves.

We create a **verified answer sheet**.

Example:

Test	Manual Agent Pass?		
Revenue growth	8.3%	8.3%	✓
AR growth	17.2%	17.2%	✓
FCF	\$X	\$X	✓
SBC	\$X	\$X	✓
Normalized NI	\$X	\$X	✓

Only after the first company passes do we test another company.

Then another industry.

20. Development stages

I suggest we build in this order:

Stage 1 — Define financial logic

We write every calculation and rule ourselves.

Stage 2 — Manual dataset

Enter one company's numbers manually.

Stage 3 — Python calculation engine

Make Python calculate everything correctly.

Stage 4 — Accuracy testing

Compare Python against our manual work.

Stage 5 — Excel generator

Automatically create the workbook.

Stage 6 — Streamlit interface

Add company inputs and results.

Stage 7 — Automatic data extraction

Connect the system to reliable financial data.

Stage 8 — AI interpretation

AI explains the already-calculated results.

Stage 9 — Automated red-flag engine

Combine signals.

Stage 10 — Full validation

Test multiple companies and difficult scenarios.

Stage 11 — Portfolio polish

README, screenshots, documentation and deployment.

21. Role of Codex

Codex should be the **developer**, not the financial analyst.

You decide:

- calculations
- accounting logic
- red-flag rules
- thresholds
- interpretation requirements
- Excel structure

Codex writes:

- Python
- data extraction
- pandas logic
- Excel automation
- Streamlit
- tests
- API connections

Before Codex codes each module, **we define exactly what that module should do.**

Then after Codex codes it:

we test it before moving forward.

22. Version strategy

We should avoid trying to build everything immediately.

V1 — Core analyzer

Manual/structured data input
Week 5 calculations
Red flags
Excel output

V2 — Automated data

Ticker input
Automatic financial extraction

V3 — AI analyst

AI interpretation
One-off classification assistance
Management-commentary analysis

V4 — Multi-company system

Peer comparisons
Sector analysis

Benchmarking
More sophisticated scoring

This keeps the first build manageable.

23. Final vision

Eventually you could enter:

MSFT

and receive:

Microsoft Earnings Quality Analysis — FY2021–FY2025

with:

financial statements → calculated metrics → working-capital analysis → cash conversion → deferred taxes → SBC → normalized earnings → red flags → analyst conclusion → downloadable Excel workbook.

The project would combine the financial-analysis knowledge you are developing with the Python, Excel and Streamlit skills already represented in your portfolio and CV.

Before we code

Do not start coding yet.

Read this plan and change anything you want: add features, remove features, change the Excel structure, change what the AI should analyze, or change the final output.

Once **you approve the blueprint**, we will turn it into a numbered build checklist and start with **Stage 1: defining the exact financial calculations and rules**.